Albertian Institute of Science & Technology

Lab Manual for

Problem Solving and Python Programming – 1131

Semester – I (CSE)

For Diploma Curriculum Revision 2026

Prepared By,

Roshni Jayan

Lecturer, Department of Computer Science & Engineering

COURSE DETAILS

ITEM	DETAILS
Programme	Diploma in Computer Engineering
Semester	Semester 1
Course Name	Problem Solving and Python Programming
Course Code	1131
Course Type	Practicum
Credits	4.5
Hours/Week	6
Total Hours	90
Curriculum	2026

Activate Windows

COURSE OUTCOME MATRIX

CO	Course Outcome	Blooms Taxonomy Level
CO1	Acquire foundational logical reasoning skills by designing flowcharts and algorithms to develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations.	A
CO2	Optimize program execution through the strategic use of control structures and loop control statements.	A
CO3	Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems	A
CO4	Design and develop modular programs using functions, modules, and packages with proper scope management and argument passing techniques	A

COURSE ARTICULATION MATRIX											
Course Outcomes	PROGRAM OUTCOMES										
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	3	2	2	2	-	-	-	-	-	3
CO2	3	3	3	2	2	-	-	-	-	-	3
CO3	3	3	3	3	3	-	-	-	-	-	3
CO4	3	2	3	3	3	-	-	-	-	-	3
Total	12	11	11	10	10	0	0	0	0	0	12
Correlation level	3	2.75	2.75	2.5	2.5	0	0	0	0	0	3
Legends: 1 - Low; 2 - Medium; 3 - High; No Correlation - "-"											

Activate Windows
Go to Settings to activate Wind

ASSESSMENT METHODOLOGY

Mode	Attendance	CE	CA1	CA2	OEE	ESE
		Continuous evaluation	Practical test (first half of experiments)	Practical test (second half of experiments)	Open-ended experiment	Practical
Duration			2 hours	2 hours		3 hours
Exam mark		50	40	40	50	40
Converted to			15	15	10	
Marks	5	30	15	10	40	Marks
Total marks	60					40
Tentative schedule	End of semester	Every lab slot	7th week	14th week	15th week	

Scheme of Evaluation - Continuous Internal Evaluation

Sl. No.	Criteria	Description	Marks
1	Preparation & Punctuality	Attendance, timely submission of record, pre-lab preparation, understanding of experiment objectives and theory.	10
2	Experimental Setup & Procedure	Ability to identify components/equipment, correct circuit connections/setup, systematic and safe execution of the procedure.	10
3	Observation & Data Recording	Accuracy and neatness of readings, correct use of units, tabulation, and systematic recording of results.	5
4	Analysis & Result Interpretation	Proper calculations, graphical representation, result verification, discussion, and logical conclusion.	10
5	Viva Voce / Concept Understanding	Clarity of theoretical concepts, explanation of procedure, ability to answer questions related to the experiment.	10
6	Workmanship, Discipline & Teamwork	Laboratory discipline, care of instruments, teamwork, attitude, and safety practices.	5
TOTAL			50

Activate Windows
Go to Settings to activate Windows

Scheme of Evaluation - Practical Test & End Semester Evaluation

Category	Things to evaluate	Marks
Write-up / Procedure (before experiment)	Aim, circuit diagram/flowchart, stepwise procedure.	10
Experiment Setup & Execution	Correct circuit connections / coding / setup. Handling of instruments/software. Accuracy of execution.	10
Observation & Result / Output	Proper tabulation of readings. Calculations/graphs/program output. Correctness of result / expected outcome.	8
Viva Voce	Conceptual understanding of experiment. Related theoretical knowledge.	8
Record	Completion & neatness of practical record. Proper certification by faculty.	4
TOTAL		40

Scheme of Evaluation - Open-ended experiment

Category	Things to evaluate	Marks
Originality and Relevance of the Idea	Assesses how creative, innovative, and relevant the chosen experiment is to the course objectives.	10
Clarity of Objectives and Experimental Plan	Evaluates how clearly the student defines the aim, scope, and systematic plan of the experiment.	10
Correctness of Execution and Data Recording	Checks the accuracy, safety, and neatness in performing the experiment and recording observations.	10
Analysis, Interpretation, and Presentation of Results	Measures the student's ability to analyze data, interpret outcomes, and present results logically.	10
Teamwork and Innovation Demonstrated	Judges the level of collaboration, initiative, and creativity shown during the experiment.	10
TOTAL		50

INDEX

Sl. No.	Title of the Practical	Mapped CO	Page No.	Executed Date	Remarks
1	Algorithm and Flowchart	CO1			
2	Installation of Python and an IDE	CO1			
3	Introduction to Python Programming	CO1			
4	Decision Control Structures	CO2			
5	Selection-based Control Structure	CO2			
6	Loop Control Structure	CO2			
7	String Manipulations	CO3			
8	List Operations	CO3			
9	Dictionary Operations	CO3			
10	Set Operations	CO3			
11	Built-in Functions	CO4			
12	User-Defined Functions	CO4			

EXPERIMENT NO: 1 Date: _____

ALGORITHM AND FLOWCHART FOR SIMPLE PROBLEMS

AIM

To understand the concepts of algorithms and flowcharts by designing step-by-step solutions for simple real-life problems before implementing them in Python.

OBJECTIVES

- To understand the concept of an algorithm.
- To learn the characteristics of a good algorithm.
- To understand common flowchart symbols and their uses.
- To develop algorithms for simple computational problems.
- To design flowcharts representing logical program execution.
- To improve logical thinking and problem-solving skills.

THEORY

Algorithm

An **algorithm** is a finite sequence of well-defined instructions used to solve a problem or perform a task. It describes the logical steps needed to obtain the desired output from the given input.

Characteristics of an Algorithm

- Simple and easy to understand
- Step-by-step procedure
- Finite number of steps
- Produces correct output
- Language independent
- Efficient and unambiguous

General Steps of an Algorithm

1. Start
2. Read/Input data
3. Perform processing
4. Make decisions (if required)
5. Repeat steps (if necessary)
6. Display output
7. Stop

Flowchart

A **flowchart** is a graphical representation of an algorithm using standard symbols connected by arrows. It helps visualize the logic of a program before coding.

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

PROBLEM STATEMENTS

1. A bank wants to develop a system to calculate the simple interest for customers based on the deposited amount, rate of interest, and time period. Design an algorithm and flowchart to calculate and display the simple interest.

2. A school wants to create a program to identify whether a student's roll number is odd or even for classroom arrangement purposes. Design an algorithm and flowchart to check whether a given number is odd or even.

3. A teacher wants to display natural numbers from 1 up to a specified limit N for teaching counting concepts in class. Design an algorithm and flowchart to print natural numbers up to N.

4. A company wants to calculate the monthly salary of its employees based on the number of working days and daily wage. Design an algorithm and flowchart to calculate and display the employee's salary.

ALGORITHM/FLOWCHART

PROBLEM 1 – SIMPLE INTEREST CALCULATION

Formula

SI = (P × R × T) / 100

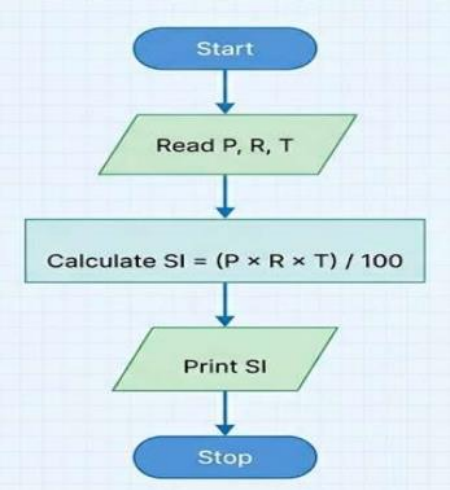
Where

- P = Principal Amount
- R = Rate of Interest
- T = Time
- SI = Simple Interest

Algorithm

1. Start
2. Read P, R and T
3. Calculate SI = (P × R × T)/100
4. Display SI
5. Stop

Flowchart

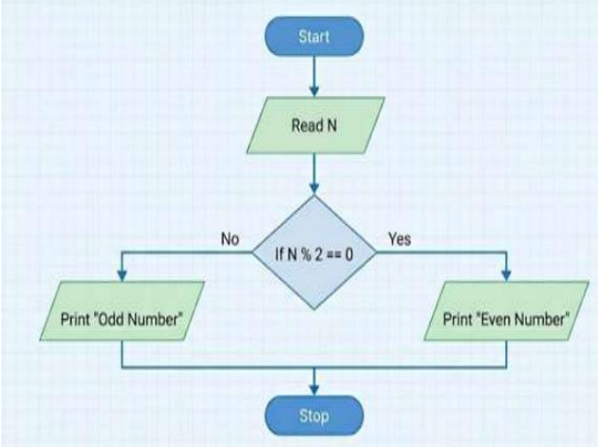


PROBLEM 2 – ODD OR EVEN

Algorithm

1. Start
2. Read N
3. If N % 2 == 0
4. Display "Even Number"
5. Else
6. Display "Odd Number"
7. Stop

Flowchart

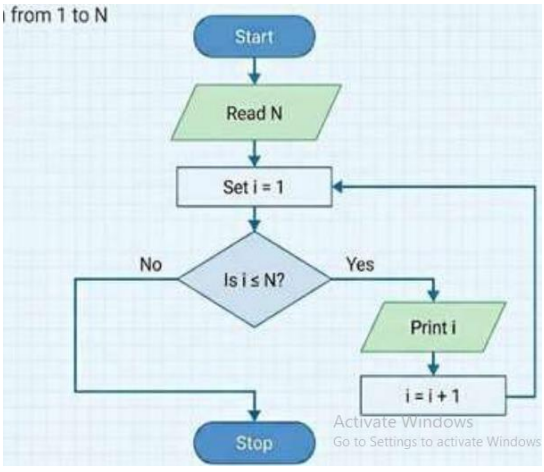


PROBLEM 3 – PRINT NATURAL NUMBERS UP TO N

Algorithm

1. Start
2. Read N
3. Set i = 1
4. While i ≤ N
5. Display i
6. i = i + 1
7. Stop

Flowchart



PROBLEM 4 – CALCULATE AND DISPLAY THE EMPLOYEE'S SALARY

Salary = Working Days × Daily Wage

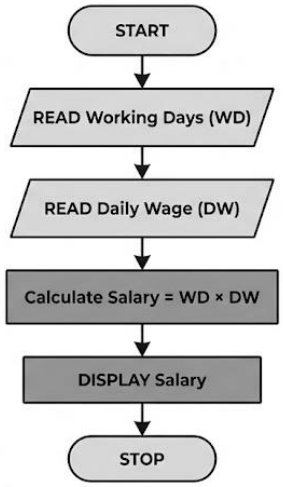
Where:

- **WD** = Number of Working Days
- **DW** = Daily Wage
- **Salary** = Monthly Salary

Algorithm

1. Start
2. Read Working Days (WD) and Daily Wage (DW)
3. Calculate Salary = WD × DW
4. Display Salary
5. Stop

Flowchart



RESULTS

The algorithm and flowchart for all problems were designed and executed successfully.

EXPERIMENT NO: 2 **Date: _____**

INSTALLATION OF PYTHON AND INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

AIM

To install Python interpreter and configure Python Integrated Development Environment (IDE) such as Visual Studio Code in Windows and Ubuntu operating systems for developing, executing, testing, and debugging Python programs.

OBJECTIVES

- To understand the Python programming environment.
- To install Python interpreter in different operating systems.
- To configure Python environment variables.
- To install and configure Visual Studio Code IDE.
- To install Python extensions required for programming.
- To execute a simple Python program and verify the installation.

THEORY

Python Interpreter

Python interpreter is a software component that executes Python programs by converting Python code into machine-understandable instructions. It provides an environment to run Python programs.

Integrated Development Environment (IDE)

An IDE is a software application that provides tools required for software development, such as:

- Code editor
- Debugger
- Program execution environment
- Syntax highlighting
- Error detection

Visual Studio Code (VS Code) is a popular lightweight IDE that supports Python programming using extensions.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel/AMD processor
- RAM: Minimum 2 GB
- Storage: Minimum 500 MB free space

Software Requirements

- Windows / Ubuntu Operating System
- Python Interpreter

- Visual Studio Code IDE

PROCEDURE

A. Installation of Python in Windows

1. Open a web browser.
2. Visit the official Python website:

https://www.python.org/downloads/

3. Download the latest Python version for Windows.
4. Open the downloaded installer.
5. Enable the option:

"Add Python to PATH"

6. Click **Install Now**.
7. Wait until the installation is completed.
8. Click **Close** after successful installation.
9. Open Command Prompt.
10. Verify Python installation using the command:

python --version

B. Installation of Python in Ubuntu

1. Open the Terminal.
2. Update the package list using:

sudo apt update

3. Install Python using:

sudo apt install python3

4. Press **Y** and press **Enter** to continue.
5. Verify Python installation using:

python3 --version

C. Installation of Python IDE (Visual Studio Code) in Windows

1. Open a web browser.
2. Visit:

https://code.visualstudio.com/

3. Download the VS Code installer for Windows.
4. Run the installer.
5. Accept the license agreement.
6. Select required options:

- Add to PATH
- Create Desktop Icon

7. Click **Install**.
8. Open Visual Studio Code after installation.
9. Install Python Extension:

- Open VS Code.
- Select **Extensions**.
- Search for **Python**.
- Install the extension developed by Microsoft.

10. Create a Python file and execute a sample program to verify the setup.

D. Installation of Python IDE (Visual Studio Code) in Ubuntu

1. Open Terminal.
2. Install VS Code using:

sudo snap install code --classic

3. Wait until installation is completed.
4. Open VS Code using:

code

5. Open the Extensions section.
6. Search and install the Python extension.
7. Create a Python file.
8. Run a sample Python program to verify the IDE configuration.

RESULT

Python interpreter and Visual Studio Code IDE were successfully installed and configured in Windows and Ubuntu operating systems. A sample Python program was executed successfully to verify the programming environment.

EXPERIMENT NO: 3
Date: _____
INTRODUCTION TO PYTHON PROGRAMMING

AIM

To write and execute basic Python programs using input, output, arithmetic operators, and variables for solving simple real-world problems.

OBJECTIVES

- To understand the basic structure of a Python program.
- To learn how to use variables and input statements.
- To perform arithmetic calculations using Python operators.
- To develop simple Python programs for real-life applications.
- To execute and test Python programs using an IDE.

PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the given problem.
4. Save the file with .py extension.
5. Run the program using the command:
   python filename.py
6. Provide the required input values.
7. Verify the output.
8. If errors occur, modify the program and execute again.
9. Test the program with different input values.
10. Record the output and result.

PROBLEM STATEMENTS

Problem 1: Temperature Conversion

Problem Statement

A weather monitoring system records temperature in degrees Celsius, but international tourists require the temperature in Fahrenheit. Write a Python program to convert temperature from Celsius to Fahrenheit and display the result.

Formula

Where:

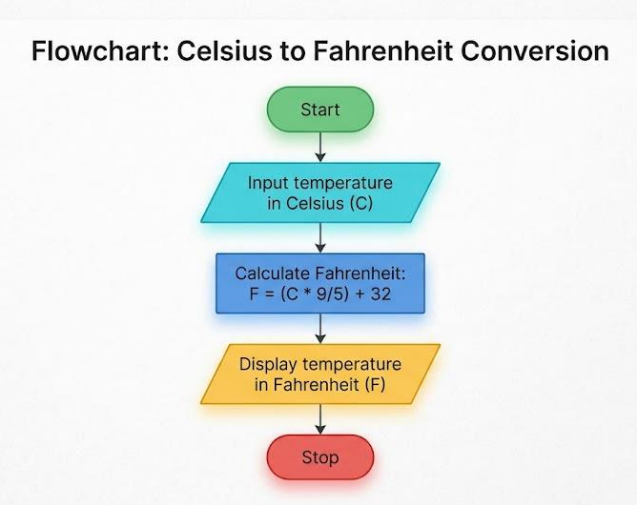
- C = Temperature in Celsius
- F = Temperature in Fahrenheit

Algorithm

1. Start
2. Input temperature in Celsius (C)
3. Calculate Fahrenheit using the formula:

F = (C × 9/5) + 32

4. Display temperature in Fahrenheit
5. Stop



Python Program

```
# Temperature conversion from Celsius to Fahrenheit

celsius = float(input("Enter temperature in Celsius: "))

fahrenheit = (celsius * 9/5) + 32

print("Temperature in Fahrenheit:", fahrenheit)
```

Sample Output

Enter temperature in Celsius: 30
Temperature in Fahrenheit: 86.0

Problem 2: Employee Salary Calculation

Problem Statement

A company wants to calculate the monthly salary of an employee based on the number of working days and daily wage. Write a Python program to calculate and display the salary.

Formula

Salary = Working Days × Daily Wage

Algorithm

1. Start
2. Input number of working days
3. Input daily wage
4. Calculate salary:

Salary = Working Days × Daily Wage

5. Display salary
6. Stop

Flowchart



Python Program

```
# Employee salary calculation

working_days = int(input("Enter number of working days: "))

daily_wage = float(input("Enter daily wage: "))

salary = working_days * daily_wage

print("Monthly Salary:", salary)
```

Sample Output

Enter number of working days: 26
Enter daily wage: 850

Monthly Salary: 22100

Problem 3: Student Percentage Calculation

Problem Statement

A school wants to calculate the percentage obtained by a student in five subjects. Write a Python program to calculate and display the percentage.

Formula

Total Marks = M1 + M2 + M3 + M4 + M5

Percentage = (Total Marks / 500) × 100

Algorithm

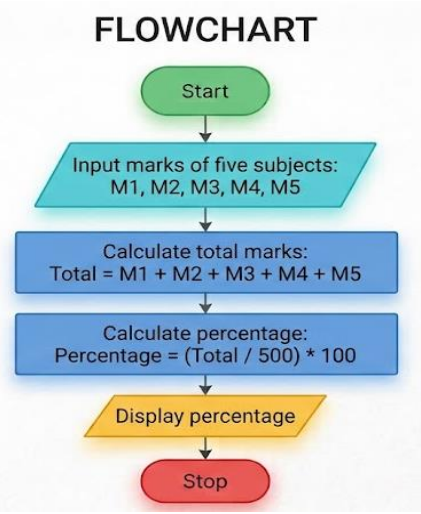
1. Start
2. Input marks of five subjects M1, M2, M3, M4 and M5
3. Calculate total marks:

Total = M1 + M2 + M3 + M4 + M5

4. Calculate percentage:

Percentage = (Total / 500) × 100

5. Display percentage
6. Stop



Python Program

```
# Student percentage calculation

m1 = float(input("Enter marks of subject 1: "))
m2 = float(input("Enter marks of subject 2: "))
```

```
m3 = float(input("Enter marks of subject 3: "))
m4 = float(input("Enter marks of subject 4: "))
m5 = float(input("Enter marks of subject 5: "))

total = m1 + m2 + m3 + m4 + m5

percentage = (total / 500) * 100

print("Total Marks:", total)
print("Percentage:", percentage)
```

Sample Output

```
Enter marks of subject 1: 80
Enter marks of subject 2: 75
Enter marks of subject 3: 90
Enter marks of subject 4: 85
Enter marks of subject 5: 70

Total Marks: 400
Percentage: 80.0
```

RESULT

The Python programs for temperature conversion, employee salary calculation, and student percentage calculation were executed successfully, and the required outputs were obtained accurately.

EXPERIMENT NO: 4 **Date: _____**

DECISION CONTROL STRUCTURES

AIM

To write and execute Python programs using decision control structures such as **if, if-else, and if-elif-else statements** to solve real-world problems.

OBJECTIVES

- To understand decision-making statements in Python.
- To learn the use of conditional expressions.
- To implement if, if-else, and nested if statements.
- To solve problems involving logical conditions.
- To develop Python programs using decision control structures.

PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the given problem.
4. Save the file with .py extension.
5. Execute the program using the command:
   python filename.py
6. Provide required input values.
7. Verify the output obtained.
8. Modify the program if errors occur.
9. Test the program with different input values.
10. Record the output and result.

PROBLEM STATEMENTS

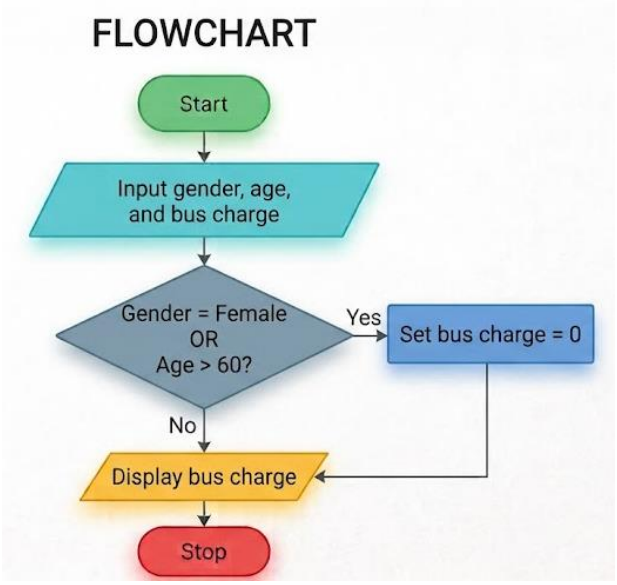
Problem 1: Bus Ticket Charge Calculation

Problem Statement

Write a Python program to input the **gender (male/female), bus charge, and age** of a passenger. If the passenger is a female or is older than 60 years, the ticket should be provided free of cost by setting the bus charge to 0. Display the charge.

Algorithm

1. Start.
2. Input gender, age, and bus charge.
3. Check whether the gender is female or age is greater than 60.
4. If the condition is true, set bus charge to 0.
5. Display the bus charge.
6. Stop.



Python Program

```
# Bus ticket charge calculation

gender = input("Enter gender (male/female): ")
age = int(input("Enter age: "))
charge = float(input("Enter bus charge: "))

if gender.lower() == "female" or age > 60:
    charge = 0

print("Bus Ticket Charge:", charge)
```

Sample Output

Enter gender (male/female): female
Enter age: 35
Enter bus charge: 50

Bus Ticket Charge: 0

Problem 2: Leap Year Checking

Problem Statement

Write a Python program to input a year from the user and determine whether it is a leap year or not.

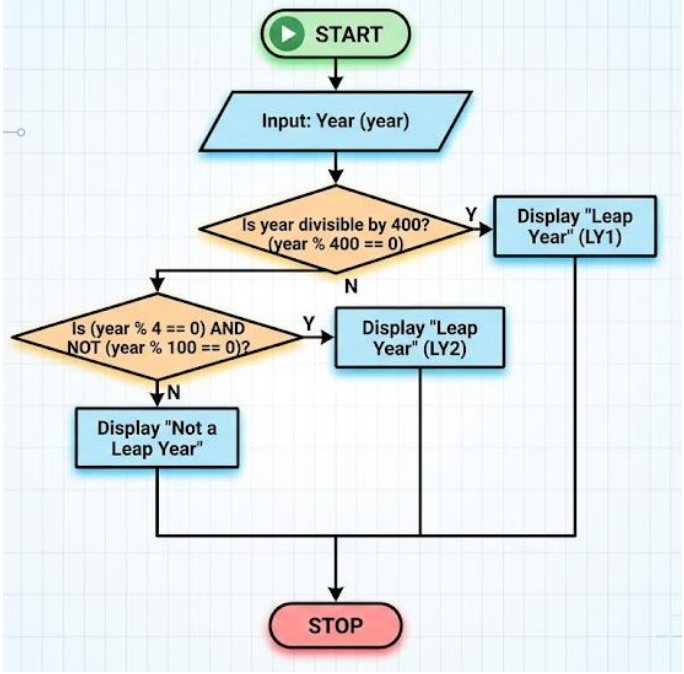
A year is a leap year if:

- It is divisible by 400, or
- It is divisible by 4 but not divisible by 100.

Display an appropriate message indicating whether the given year is a leap year or a common year.

Algorithm

1. Start.
2. Input a year.
3. Check whether the year is divisible by 400.
4. If yes, display "Leap Year".
5. Else check whether the year is divisible by 4 and not divisible by 100.
6. If yes, display "Leap Year".
7. Otherwise display "Not a Leap Year".
8. Stop.



Python Program

```
# Leap year checking program

year = int(input("Enter a year: "))

if year % 400 == 0:
    print("Leap Year")
elif year % 100 == 0:
    print("Not a Leap Year")
elif year % 4 == 0:
    print("Leap Year")
else:
    print("Not a Leap Year")
```

Sample Output

```
Enter a year: 2024

Leap Year
```

Problem 3: Electricity Bill Calculation

Problem Statement

Write a Python program to input the number of electricity units consumed by a customer and calculate the electricity bill.

Billing rules:

- Up to 100 units → Rs. 5 per unit
- More than 100 and up to 300 units → Rs. 8 per unit
- Above 300 units → Rs. 10 per unit

Display the total electricity bill amount.

Algorithm

1. Start.
2. Input electricity units consumed.
3. If units ≤ 100, calculate:

Bill = Units × 5

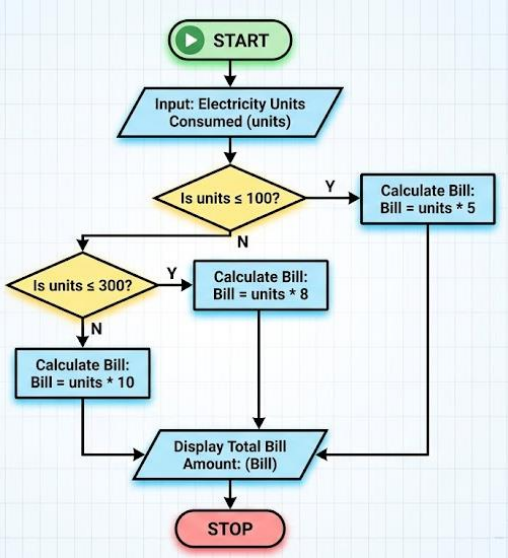
4. Else if units ≤ 300, calculate:

Bill = Units × 8

5. Otherwise calculate:

Bill = Units × 10

6. Display total bill amount.
7. Stop.



Python Program

```
# Electricity bill calculation

units = int(input("Enter electricity units consumed: "))

if units <= 100:
    bill = units * 5

elif units <= 300:
    bill = units * 8

else:
    bill = units * 10

print("Electricity Bill Amount: Rs.", bill)
```

Sample Output

Enter electricity units consumed: 250

Electricity Bill Amount: Rs. 2000

RESULT

The Python programs using decision control structures were executed successfully, and the required outputs were obtained accurately for the given inputs.

EXPERIMENT NO: 5 **Date: _____**

SELECTION BASED CONTROL STRUCTURE

AIM

To write and execute Python programs using selection-based control structures such as **if-elif-else ladder and match-case statements** to solve decision-making problems.

OBJECTIVES

- To understand selection-based control structures in Python.
- To implement multiple condition checking using if-elif-else statements.
- To learn the usage of match-case statements.
- To develop programs involving decision-making logic.
- To execute and test Python programs with different inputs.

THEORY

Selection Based Control Structure

Selection control structures allow a program to execute different blocks of code based on whether a given condition is true or false.

Python provides different selection statements:

1. if Statement

Used to execute a block of code only when a condition is true.

2. if-else Statement

Used when there are two possible choices.

3. if-elif-else Ladder

Used when multiple conditions need to be checked.

4. match-case Statement

Introduced in Python 3.10, it is used for pattern matching and selecting one option from multiple choices.

PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the given problem.
4. Save the file with .py extension.

Example:

grade.py

5. Run the program using the command:

python filename.py

6. Provide the required input values.
7. Verify that the output is correct.
8. If errors occur, modify the program and execute again.

9. Test the program with different input values.
10. Record the output and result.

PROBLEM STATEMENTS

Problem 1: Student Grade Evaluation

Problem Statement

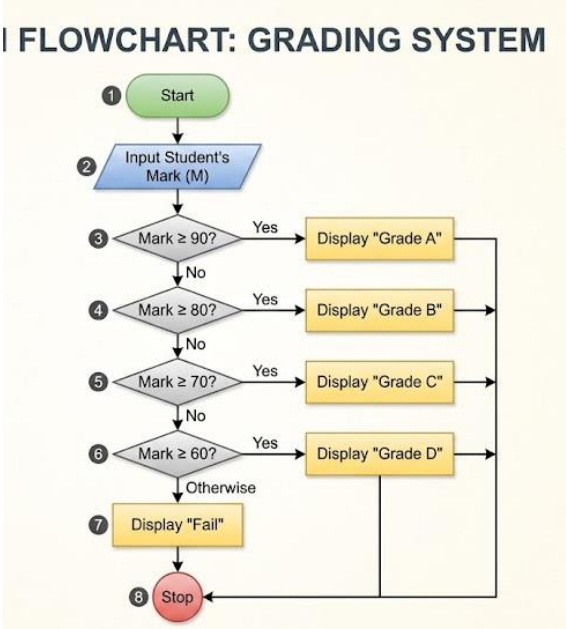
A school wants to automate the process of assigning grades to students based on their examination marks. The system should accept a student's mark and display the appropriate grade according to the school's grading policy.

Grading Policy

Marks	Grade
90 and above	A
80 – 89	B
70 – 79	C
60 – 69	D
Below 60	Fail

Algorithm

1. Start.
2. Input student's mark.
3. If mark ≥ 90, display Grade A.
4. Else if mark ≥ 80, display Grade B.
5. Else if mark ≥ 70, display Grade C.
6. Else if mark ≥ 60, display Grade D.
7. Otherwise display Fail.
8. Stop.



Python Program

```
# Student Grade Evaluation

mark = int(input("Enter student's mark: "))

if mark >= 90:
    grade = "A"

elif mark >= 80:
    grade = "B"
```

```
elif mark >= 70:
    grade = "C"

elif mark >= 60:
    grade = "D"

else:
    grade = "Fail"

print("Grade:", grade)
```

Sample Output

Enter student's mark: 85

Grade: B

Problem 2: Day Name Display

Problem Statement

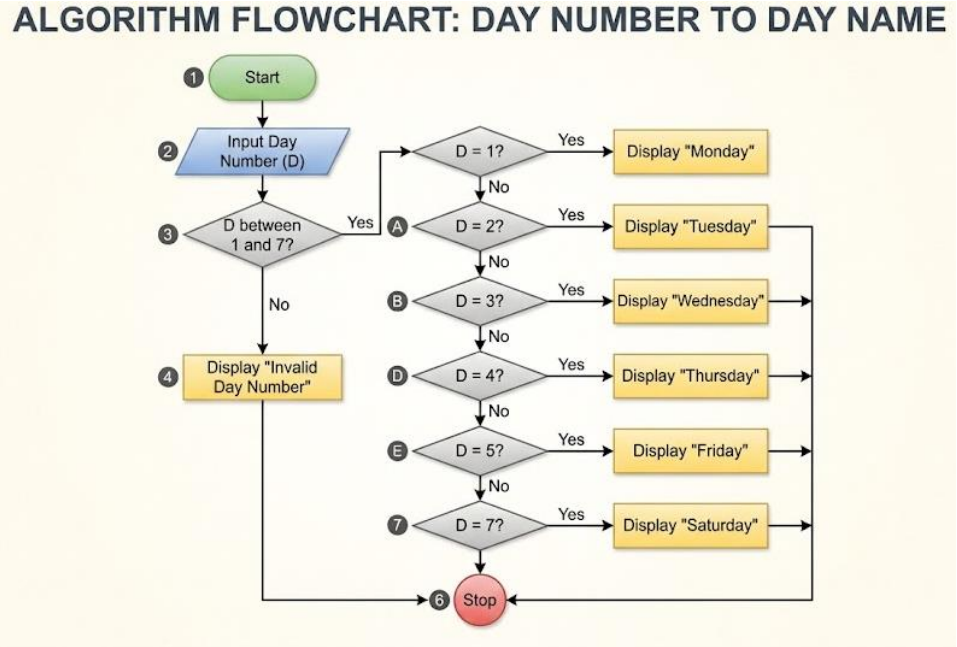
A digital calendar application needs to display the day name when a user enters a day number (1–7). The program should match the entered number with the corresponding day.

Day Number Mapping

Day Number	Day
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

Algorithm

1. Start.
2. Input day number.
3. Match the day number with the corresponding day.
4. Display the day name.
5. If the number is not between 1 and 7, display "Invalid Day Number".
6. Stop.



Python Program

```
# Day Name Display using match-case

day = int(input("Enter day number (1-7): "))

match day:

    case 1:
        print("Sunday")

    case 2:
        print("Monday")

    case 3:
        print("Tuesday")

    case 4:
        print("Wednesday")

    case 5:
        print("Thursday")

    case 6:
        print("Friday")

    case 7:
        print("Saturday")
```

```
    case _:
        print("Invalid Day Number")
```

Sample Output

Enter day number (1-7): 4

Wednesday

RESULT

The Python programs using selection-based control structures such as **if-elif-else ladder and match-case statements** were successfully designed, implemented, and executed. The desired outputs were obtained for different inputs.